Ejercicios APE 5 – Manejo de ficheros en Java

Estudiante: Antony Sebastian Casillas Ochoa

Universidad Técnica de Ambato – Carrera de Software

Ejercicio 7. Exploración y gestión de metadatos con File

Enunciado:
Desarrollar un programa en Java que interactúe con la ruta **datos/prueba.txt**. Primero debe comprobar si el archivo existe; si no existe, debe crearlo controlando la excepción IOException. Finalmente, debe mostrar el nombre y la ruta utilizada.

Explicación de la solución:

Se crea un objeto File con la ruta indicada y se utiliza exists() para comprobar si el archivo está disponible. Si no existe, se crea la carpeta contenedora con mkdirs() y luego se intenta crear el archivo mediante createNewFile(). La operación se encuentra dentro de un bloque try-catch para controlar IOException. Al final se muestran getName() y getPath().

Código fuente (Ejercicio7.java):

```
package Ape5;

import java.io.File;
import java.io.IOException;

public class Ejercicio7 {

    public static void main(String[] args) {

        File archivo = new File("datos/prueba.txt");

        try {
            if (archivo.exists()) {
                System.out.println("El archivo ya existe.");
            } else {
                archivo.getParentFile().mkdirs();

                if (archivo.createNewFile()) {
                    System.out.println("Archivo creado correctamente.");
                }
            }

            System.out.println("Nombre: " + archivo.getName());
            System.out.println("Ruta: " + archivo.getPath());

        } catch (IOException e) {
            System.out.println("Error al crear el archivo: "
                    + e.getMessage());
        }
    }
}
```

Resultado de la ejecución:

```
Archivo creado correctamente.
Nombre: prueba.txt
Ruta: datos\\prueba.txt
```

Ejercicio 8. Escritura estructurada por líneas con BufferedWriter

Enunciado:
Crear un archivo de texto llamado **materias.txt** y escribir cinco asignaturas elegidas. Se debe utilizar BufferedWriter enlazado con FileWriter, separar cada asignatura con newLine(), cerrar el flujo y controlar IOException.

Explicación de la solución:

Se crea un BufferedWriter utilizando un FileWriter con el nombre materias.txt. Después se escriben cinco asignaturas mediante write(), colocando newLine() después de cada una para que queden en líneas independientes. Finalmente, se cierra el flujo con close(). El bloque try-catch permite controlar posibles errores durante la escritura.

Código fuente (Ejercicio8.java):

```
package Ape5;

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class Ejercicio8 {

    public static void main(String[] args) {

        try {
            BufferedWriter escritor = new BufferedWriter(
                    new FileWriter("materias.txt")
            );

            escritor.write("Programacion");
            escritor.newLine();

            escritor.write("Algebra");
            escritor.newLine();

            escritor.write("Fisica");
            escritor.newLine();

            escritor.write("Calculo");
            escritor.newLine();

            escritor.write("Metodologia de la Investigacion");
            escritor.newLine();

            escritor.close();

            System.out.println(
                    "Archivo creado y materias guardadas correctamente."
            );

        } catch (IOException e) {
            System.out.println("Error al escribir el archivo: "
                    + e.getMessage());
        }
    }
}
```

Resultado de la ejecución:

```
Archivo creado y materias guardadas correctamente.

Contenido de materias.txt:
Programacion
Algebra
Fisica
Calculo
Metodologia de la Investigacion
```

Ejercicio 9. Control del modo de apertura (Append vs Sobrescribir)

Enunciado:
A partir del archivo materias.txt creado en el ejercicio anterior, abrirlo en modo de adición (append = true) y agregar dos nuevas asignaturas al final, utilizando newLine() para mantener una asignatura por línea. Se debe cerrar el flujo y comprobar que las cinco asignaturas anteriores se conserven.

Explicación de la solución:

Se utiliza FileWriter con el segundo parámetro establecido en true. Esto activa el modo append, que permite añadir información al final del archivo sin eliminar el contenido existente. Con BufferedWriter se escriben las dos nuevas asignaturas y se utiliza newLine() después de cada una. Al terminar, se cierra el flujo con close().

Código fuente (Ejercicio9.java):

```
package Ape5;

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class Ejercicio9 {

    public static void main(String[] args) {

        try {
            BufferedWriter escritor = new BufferedWriter(
                    new FileWriter("materias.txt", true)
            );

            escritor.write("Programacion Orientada a Objetos");
            escritor.newLine();

            escritor.write("Estructura de Datos");
            escritor.newLine();

            escritor.close();

            System.out.println(
                    "Nuevas materias agregadas correctamente."
            );

        } catch (IOException e) {
            System.out.println("Error al agregar materias: "
                    + e.getMessage());
        }
    }
}
```

Resultado de la ejecución:

```
Nuevas materias agregadas correctamente.

Contenido de materias.txt:
Programacion
Algebra
Fisica
Calculo
Metodologia de la Investigacion
Programacion Orientada a Objetos
Estructura de Datos
```

Ejercicio 10. Ciclo de vida completo y borrado seguro

Enunciado:
Crear un programa que represente la ruta **datos/temporal.txt**, compruebe si existe y, si no está disponible, lo cree controlando IOException. Luego debe utilizar una variable booleana para simular la confirmación del usuario y ejecutar delete() únicamente cuando la confirmación sea verdadera. Se debe mostrar el resultado de la eliminación.

Explicación de la solución:

Se construye un objeto File con la ruta datos/temporal.txt. Mediante exists() se comprueba si el archivo ya existe; de lo contrario, se crea primero la carpeta y después el fichero con createNewFile(). La variable confirmarBorrado representa la autorización para eliminarlo. Cuando su valor es true, se ejecuta delete(), cuyo resultado booleano indica si la eliminación se realizó correctamente. La posible IOException se controla con try-catch.

Código fuente (Ejercicio10.java):

```
package Ape5;

import java.io.File;
import java.io.IOException;

public class Ejercicio10 {

    public static void main(String[] args) {

        File archivo = new File("datos/temporal.txt");

        try {
            if (archivo.exists()) {
                System.out.println("El archivo ya existe.");
            } else {
                archivo.getParentFile().mkdirs();

                if (archivo.createNewFile()) {
                    System.out.println("Archivo creado correctamente.");
                }
            }

            boolean confirmarBorrado = true;

            if (confirmarBorrado) {
                boolean eliminado = archivo.delete();

                System.out.println(
                        "¿Se eliminó el archivo?: " + eliminado
                );
            } else {
                System.out.println(
                        "Se canceló la eliminación del archivo."
                );
            }

        } catch (IOException e) {
            System.out.println("Error al gestionar el archivo: "
                    + e.getMessage());
        }
    }
}
```

Resultado de la ejecución:

```
Archivo creado correctamente.
¿Se eliminó el archivo?: true
```

Conclusiones

1. La clase File permite representar rutas de archivos y consultar su estado mediante métodos como exists(), getName() y getPath(). También permite crear y eliminar archivos con createNewFile() y delete().

2. La combinación de FileWriter y BufferedWriter facilita la escritura de información en archivos de texto. El método newLine() permite organizar el contenido para que cada asignatura quede en una línea independiente.

3. El parámetro append = true permite agregar información al final de un archivo sin sobrescribir los datos anteriores, mientras que el cierre del flujo ayuda a liberar los recursos utilizados.

4. Antes de eliminar un archivo es importante comprobar su existencia y validar la confirmación. Además, el manejo de IOException permite responder de forma controlada ante posibles errores de entrada y salida.

Verificación general:

Los ejercicios se deben ejecutar desde el mismo proyecto para que materias.txt sea compartido por los ejercicios 8 y 9. Primero se ejecuta el ejercicio 8 y luego el 9; así se puede comprobar que las dos asignaturas nuevas se agregan a las cinco originales. Los ejercicios 7 y 10 trabajan con archivos dentro de la carpeta datos.